

UNIVERSIDADE FEDERAL DE MINAS GERAIS - UFMG

Trabalho Final de Automação em Tempo Real

Etapa 1 — Arquitetura e implementação parcial do sistema de logística com robôs móveis

LUCAS ARAÚJO PINTO RESENDE **2021013833**

ENZO VÍDERO PINHEIRO BAHIA **2022061580**

JOÃO PAULO GONÇALVES DA SILVA **2023422633**

1. Introdução

Este trabalho tem como objetivo desenvolver um sistema de gerenciamento e simulação de uma frota de robôs móveis autônomos (AMRs), responsável por organizar o transporte de cargas em um ambiente logístico.

O sistema deverá coordenar os robôs disponíveis, determinar rotas até as baias de recebimento e os destinos das cargas e sincronizar a atuação de múltiplos robôs quando uma tarefa exigir transporte colaborativo.

Nesta primeira etapa, o foco está na definição da arquitetura de software, na implementação dos módulos centrais e na comunicação entre processos e threads. As interfaces gráficas e a comunicação via MQTT serão desenvolvidas na segunda etapa.

2. Arquitetura do sistema

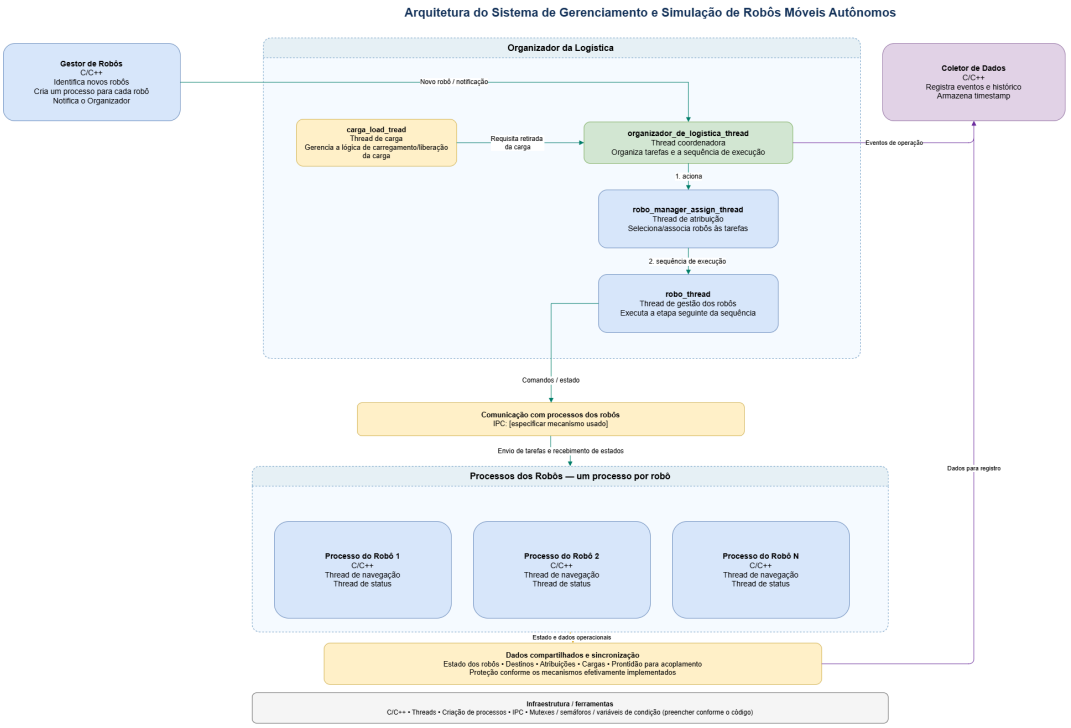
2.1 Visão geral

O sistema foi dividido em módulos com responsabilidades específicas, de modo a separar o gerenciamento dos robôs, a organização das tarefas, a navegação de cada robô, o monitoramento dos recursos compartilhados e o registro dos eventos.

Os principais módulos previstos são:

- **Gestor de Robôs:** responsável por identificar novos robôs e iniciar seus processos.
- **Organizador da Logística:** responsável por distribuir tarefas, selecionar cargas e coordenar a execução das missões.
- **Processo do Robô:** responsável pela execução das tarefas atribuídas a cada robô assim como sua navegação.
- **Coletor de Dados:** responsável por registrar eventos e informações relevantes para a rastreabilidade do sistema.

A Figura abaixo apresenta a arquitetura geral do sistema, incluindo seus mecanismos de comunicação e sincronização adotados.



2.2 Tecnologias utilizadas

O desenvolvimento dos módulos de gerenciamento e dos processos dos robôs é realizado em C/C++.

As bibliotecas, APIs e ferramentas utilizadas para criação de processos, gerenciamento de threads, comunicação entre módulos e sincronização devem ser especificadas nesta seção.

- Linguagem de programação: C++.
- Gerenciamento de threads: POSIX.
- Comunicação entre processos (IPC): Pipes e memória compartilhada.
- Sincronização de dados compartilhados: POSIX e bibliotecas nativas do C++.
- Compilação: Makefile.

3. Processos e threads

3.1 Organizador da Logística

O Organizador da Logística é responsável por distribuir as tarefas de transporte e acompanhar a disponibilidade dos robôs. Para cada missão, deve determinar os robôs participantes, a carga a ser transportada, a baía de origem e o destino.

No caso de transporte colaborativo, a operação da carga somente deve começar quando todos os robôs necessários estiverem posicionados e prontos para executar a tarefa.

Para sua implementação, foram pensados 4 tipos de *threads* principais:

- ***Carga_load_thread:*** Simula uma zona de carga, onde chegam os objetos a serem transportados pelos robôs. São criadas threads para cada baia de carga que compõe o sistema.
- ***Organizador_de_logistica_thread:*** Simula a tomada de decisão de mover uma carga que chegou no sistema pela *Carga_load_thread.* Recebe como parâmetro uma Baia de carga e o gerenciador de robôs para que ao chegar a carga consiga alocar outra thread para gerenciar as trajetórias dos robôs necessários.
- ***Robo_manager_assign_thread:*** Simula todos os aspectos que envolvem a "missão" de uma quantidade certa de robôs serem destinados a baia de carga, todos ao mesmo tempo a agruparem e destinam tal carga até a zona de descarga. Essa thread é criada por uma *Robo_manager_assign_thread* cada vez que for necessário tratar esse envio.
- ***Robo_thread:*** Simula a movimentação do robô pelo sistema, de acordo com os dados coletados pelo processo do robô. Cada thread dessa é criada dentro do contexto da *Robo_manager_assign_thread* para cada direção que é necessária para que um móvel autônomo se dirija.

Todas as classes que são criadas para a representação do sistema apresentam seus objetos de sincronização e para exclusão mútua no caso de múltiplos acessos simultâneos.

3.2 Processo e threads de cada robô

Cada robô é gerenciado por um processo próprio, criado pelo Gestor de Robôs com fork + exec e executado como robo <id> <slot> <x> <y>. Ao iniciar, o processo se conecta à memória compartilhada POSIX da frota (/atr_frota) e abre a sua fila de mensagens de comandos (/atr_robo_<id>_cmd). Os dois recursos são criados pelo Gestor antes do fork, para que o Organizador possa enviar comandos assim que o robô é anunciado.

O processo tem três threads principais. A thread principal bloqueia os sinais SIGINT/SIGTERM, trata-os de forma síncrona com sigwait e coordena o encerramento ordenado. A thread de Comando de Navegação é periódica, com período de 50 ms e ativação absoluta por clock_nanosleep. A cada ciclo ela esvazia a fila de comandos sem bloquear, calcula a rota pelo grafo do galpão (Dijkstra) e envia os setpoints de posição ao hardware, waypoint por waypoint. Antes de cada passagem estreita, pede vaga a um monitor na memória compartilhada, com capacidade N, fila FIFO e reserva para equipes; se não houver vaga, aguarda parada na entrada, sem bloquear a thread. Essa thread também executa a tarefa de transporte em fases (coleta, espera da equipe, acoplamento sincronizado, transporte e entrega), mantendo a equipe unida por uma barreira por waypoint lida do status dos colegas.

A thread de Status do Robô é periódica, com período de 100 ms, e lê as entradas do hardware. Ela detecta defeitos, bloqueando o movimento e colocando o robô no estado "defeito", e monitora a bateria: abaixo de 20%, reserva uma vaga numa estação de recarga e força a navegação até ela, deixando o robô indisponível até 95%. Também acumula a distância total percorrida e publica o status no slot do robô na tabela compartilhada, protegido por um mutex PTHREAD_PROCESS_SHARED e PTHREAD_MUTEX_ROBUST, que o Organizador lê.

Os eventos com timestamp vão para um buffer circular produtor/consumidor com semáforos, consumido pelo Coletor de Dados; o robô usa sem_trywait para nunca bloquear esperando o Coletor. As duas threads trocam informações por uma estrutura de contexto protegida por mutex: o Status pode bloquear o movimento ou impor um destino, e a Navegação publica a fase da tarefa e o estado operacional desejado

3.3 Coletor de Dados

O Coletor de Dados registra em disco, com timestamp de data, hora e milissegundos, tudo o que acontece na frota, para fins de rastreabilidade. Ele consome o buffer circular de eventos na memória compartilhada, no qual os processos dos robôs publicam sem bloquear, e é organizado em duas threads separadas por uma fila interna.

A thread de coleta aguarda no semáforo do buffer (sem_timedwait) e apenas transfere cada evento para a fila interna, sem fazer E/S, para que um disco lento nunca faça o buffer encher e os robôs descartarem eventos.

A thread de gravação, sincronizada por mutex e variável de condição, retira a fila inteira de uma vez e grava o histórico de cada robô em historico_robo_<id>.log, com um arquivo por robô. Um módulo de rastreabilidade interpreta os eventos de carga (“pegou carga na Baia 3”, “entregou carga na Zona C”) e escreve em cargas.csv uma linha por carga, com baia, destino, robôs que a transportaram e horários de coleta e entrega.

A carga só é fechada quando todos os robôs que a pegaram a entregaram, e as cargas interrompidas, por exemplo por um robô que morreu no caminho, são gravadas como incompletas no encerramento.

Cada registro também é enviado, sem bloquear, por uma fila de mensagens POSIX ao Organizador da Logística, que pode repassá-lo à Tela de Operação via MQTT. Ao encerrar, o Coletor imprime estatísticas de perdas, uso máximo da fila interna e latência máxima até a gravação, que servem de base para a análise de escalonabilidade.

O módulo foi validado por um teste de integração automatizado (make teste-coletor), que verifica o histórico por robô, a rastreabilidade de cargas, o envio ao Organizador, a carga incompleta e uma rajada de 20 robôs simultâneos sem perda de eventos.

4. Testes realizados

Os testes desta etapa têm como finalidade verificar a comunicação entre os módulos e o comportamento dos mecanismos de sincronização.

4.1 Gestor de Robôs e Processo do Robô

Os testes do Gestor de Robôs e do processo do robô são de integração e executam o sistema real, com processos separados, filas de mensagens POSIX, memória compartilhada e threads periódicas. Como o Organizador, o Coletor e o simulador ainda não existem nesta etapa, ferramentas de linha de comando assumem esses papéis: novo_robo

anuncia robôs ao Gestor, comando_robo envia comandos e injeta falhas ou altera a bateria, escuta_gestor consome os eventos ROBO_NOVO e ROBO_REMOVIDO e monitor_frota lê a tabela de status, as passagens, as vagas de recarga e o buffer de eventos. Cada teste dispara uma ação e aguarda a condição esperada com prazo máximo, em quatro scripts executados com make teste.

No teste do Gestor (14 verificações), confere-se que cada anúncio gera um processo filho e um evento ROBO_NOVO, que ids repetidos e mensagens malformadas são recusados sem derrubar o Gestor e que um robô morto com kill -9 gera ROBO_REMOVIDO e pode ser recriado. Verifica-se também que 20 anúncios simultâneos não provocam perda de mensagens e que o encerramento por SIGTERM termina com código 0, sem processos órfãos.

No teste do processo do robô (27 verificações), valida-se o caminho completo de um comando IR_PARA, da fila de comandos até o hardware e o status, terminando com o evento CHEGOU_DESTINO. Verificam-se ainda os modos manual e automático, a publicação de falhas, bateria e acoplamento no status e, após a morte de um robô, a liberação do slot e da fila sem afetar os demais.

No teste da navegação (35 verificações), confirma-se que as rotas respeitam a capacidade das passagens conforme o tamanho da equipe, que a passagem de capacidade 1 atende os robôs em ordem FIFO, um por vez, e que a de capacidade 2 admite dois robôs simultâneos. No transporte em equipe da Baia 3 até a Zona C, os dois robôs só partem após o acoplamento, atravessam a passagem juntos e mantêm distância máxima de 8 unidades entre si. Testa-se também a liberação da vaga quando um robô morre ou recebe PARAR dentro da passagem. Um processo amostrou a ocupação das passagens a cada 50 ms e, em cerca de 1.490 amostras, ela nunca excedeu a capacidade.

No teste de status (50 verificações), confere-se a distância acumulada, a parada imediata em caso de defeito e a retomada do trajeto após a remoção da falha. Com bateria baixa, o robô reserva uma vaga, recarrega até 95% e só então retoma a tarefa recebida; quando a Recarga 1 lota, o quarto robô é direcionado à Recarga 2, e um robô carregando caixa adia a recarga até a entrega. Verificam-se ainda o aviso sem recarga forçada no modo manual, o estado de defeito com bateria em 0% e a liberação da vaga de um robô morto.

Os quatro scripts passaram em três a cinco execuções consecutivas, sem falhas. Manualmente, verificou-se também a recuperação do mutex robusto (EOWNERDEAD) após um processo terminar com o slot travado e o encerramento de todos os robôs quando o Gestor é morto, via PR_SET_PDEATHSIG. A criação de um robô levou de 100 a 350 µs, e o pior tempo de execução por ciclo foi de cerca de 90 µs na navegação e 35 µs no status, sem nenhum prazo perdido. Esses valores, medidos nesta máquina, servem de ponto de partida para a análise de escalonabilidade da Etapa 2.

4.2 Coletor de Dados

Para validar o Coletor de Dados, foi desenvolvido um teste de integração automatizado (make teste-coletor) que inicia o Gestor de Robôs, o Coletor e um leitor da fila POSIX que simula o Organizador da Logística, cria robôs simulados e confere os arquivos gerados em oito cenários.

No primeiro, verifica-se que cada robô tem seu arquivo de histórico, que toda linha possui timestamp com data, hora e milissegundos, que os eventos estão em ordem cronológica e que o histórico de um robô não se mistura com o de outro. No segundo, confere-se o registro de defeito, de bateria baixa e de recarga. No terceiro, valida-se a rastreabilidade com uma equipe de dois robôs transportando uma carga da Baia 3 até a Zona C, verificando que a carga só é registrada após a entrega e que cargas.csv recebe uma única linha com baia, destino, robôs e horários corretos. No quarto, confirma-se a chegada dos eventos e dos registros de carga ao Organizador pela fila POSIX. No quinto, testa-se a persistência: os dados permanecem íntegros após um kill -9 do Coletor e uma nova execução acrescenta aos arquivos existentes, sem duplicar o cabeçalho nem sobrescrever o histórico. No sexto, uma rajada de 20 robôs criados simultaneamente não provoca nenhum descarte de evento no buffer compartilhado. No sétimo, a morte de um robô durante o transporte resulta no registro da carga como incompleta. No oitavo, o encerramento por SIGTERM termina com código 0, sem erros de gravação e sem perdas na fila interna.

O teste passou em três execuções consecutivas, em um ambiente com um único núcleo de CPU, com 146 eventos gravados, no máximo 12 eventos acumulados na fila interna e latência máxima de cerca de 8,7 ms entre a publicação do evento pelo robô e sua gravação em disco.

```
./testes/teste_coletor.sh
1. Histórico por robô com timestamp
  [OK]    histórico do robô 1 criado com evento INICIADO
  [OK]    histórico do robô 2 criado
  [OK]    evento CHEGOU_DESTINO no histórico
  [OK]    todas as linhas têm data, hora e ms
  [OK]    eventos em ordem cronológica
  [OK]    histórico de um robô não mistura outro
2. Defeito, bateria e recarga no histórico
  [OK]    defeito presente registrado
  [OK]    defeito removido registrado
  [OK]    recarga registrada
  [OK]    recarga concluída registrada
3. Rastreabilidade: Baia 3 -> Zona C com robôs 8 e 9
  [OK]    robôs 8 e 9 prontos
  [OK]    robôs na coleta
  [OK]    robô 8 na coleta
  [OK]    equipe transportando
  [OK]    carga ainda em transporte não aparece como entregue
  [OK]    robô 8 concluiu a entrega
  [OK]    carga 42 registrada em cargas.csv
  [OK]    baia, destino e robôs corretos (Baia 3 -> Zona C, robôs 8+9)
  [OK]    uma única linha por carga (não uma por robô)
  [OK]    eventos 'pegou' e 'entregou' no histórico dos robôs
  [OK]    cabeçalho do cargas.csv
4. Disponibilização ao Organizador
  [OK]    evento do robô 3 chegou ao Organizador
  [OK]    robô 4 na coleta
  [OK]    robô 5 na coleta
  [OK]    registro de carga chegou ao Organizador
```

```
5. Persistência: dados sobrevivem a kill -9 e o histórico continua depois
  [OK]    histórico em disco após kill -9 (6 linhas)
  [OK]    rastreabilidade em disco após kill -9
  [OK]    novo Coletor acrescenta ao histórico existente
  [OK]    cabeçalho do cargas.csv não duplicado
  [OK]    histórico anterior preservado (não sobrescrito)
6. Rajada: 20 robôs ao mesmo tempo sem perder eventos
  [OK]    20 históricos criados, um por robô (INICIADO)
  [OK]    evento CHEGOU_DESTINO dos 20 robôs registrado
  [OK]    nenhum evento descartado no buffer compartilhado
7. Carga incompleta: robô morre durante o transporte
  [OK]    robôs 12 e 13 prontos
  [OK]    robô 12 na coleta
  [OK]    robô 13 na coleta
  [OK]    equipe transportando
  [OK]    robô 12 pegou a carga
  [OK]    robô 13 pegou a carga
8. Encerramento ordenado
  [OK]    Coletor saiu com código 0
  [OK]    carga 55 gravada como incompleta (robô morreu no caminho)
  [OK]    encerrou sem erros de gravação
  [OK]    nenhuma perda no buffer compartilhado nem na fila interna
[COLETOR] encerrado: 146 eventos gravados, 1 cargas registradas | perdas: 0 no
 buffer compartilhado, 0 na fila interna, 3 na fila do Organizador, 0 erros de
 gravação | fila interna máx. 16, latência máx. até gravar 11247 us

Logs em run/teste_coletor/ e dados em run/teste_coletor/dados/
TODOS OS TESTES PASSARAM
```

5. Conclusão

Com os testes necessários realizados, foi observado que cada divisão do código se mostrou suficiente individualmente, faltando apenas a integração final entre todas as partes e a adição dos processos restantes da etapa 2, como o MQTT, a interface gráfica e a simulação do sistema físico do robô.